```
---
title: "Interlimb Asymmetry"
author: "Samuel Montalvo, Ph.D."
date: "2024-01-06"
output: html_document
---

```{r setup, include=FALSE}
library(readr)
library(readxl)
library(tidyverse)
library(ggprism)
library(ggpubr)
library(lme4)
library(lmerTest)
library(sjPlot)
library(table1)
library(lubridate)
library(janitor)
library(easystats)
library(gtsummary)
library(ggeffects)

```


Intro

```{r, warning=FALSE, message=FALSE}
df <- read_csv("Wushu_Malaysia_1yr_Jumps.csv")

df <- df %>% select_if(~ !all(is.na(.)))

df <- df %>% select(-TestId)

#cleans column names
df <- janitor::clean_names(df)


# Date manipulation
# Ensure that the Date is in the correct Date format
# Assuming the date format in your data is day/month/year
df$date <- dmy(df$date)

# Remove rows with NA dates
df <- df[!is.na(df$date), ]

# Create a new variable with year and month, and convert it to a factor
df$month <- format(df$date, "%Y-%m")
df$month <- factor(df$month)

#training session
df$session_id <- interaction(df$name, df$date)


# variables to cointiinous
df$left_avg_propulsive_force <- as.numeric(df$left_avg_propulsive_force)
df$right_avg_propulsive_force <- as.numeric(df$right_avg_propulsive_force)

df$week <- isoweek(df$date)

df
```

```{r, message=FALSE}
```
```

```
Demographics <- read_csv("Demographics.csv")

Demographics <- janitor::clean_names(Demographics)

Demographics <- Demographics %>% rename(name = athlete, DOB = date_of_birth_dd_mm_yy) %>%
  select(name, DOB, sex, height)

Demographics$DOB <- dmy(Demographics$DOB)


Demographics
```

```{r}
df2 <- inner_join(df, Demographics, by=c("name"))

df2
```

```{r}
names(df)
```

```{r}
library(lubridate)


# Convert date to Date format (if it's not already)
df2$date <- as.Date(df2$date)

# Calculate age, handling NA values in DOB
df2$Age <- ifelse(!is.na(df2$DOB), interval(df2$DOB, df2$date) / years(1), NA)

# View the updated data frame with selected columns
df2 %>% select(date, name, DOB, Age)
```

```{r}
df3 <- inner_join(df, Demographics, by=c("name"))
df3
```

```{r}
table1(~ type + tags,data=df3)
```

```{r}
df <- df2
```

```{r}
df <- df %>% filter(type == "Countermovement Jump")

df <- df[is.na(df$tags) | df$tags == '' | df$tags == 'Pre', ]

df
```

```{r}
df <- df %>% mutate(mass = system_weight/9.81, jump_height_cm = jump_height*100)

first_observation_df <- df %>%
  group_by(name) %>%
  slice(1)
```

```
table1(~ week + Age + mass + jump_height_cm + m_rsi | sex, data=first_observation_df)
```

```{r}
table1 <- first_observation_df %>%
  ungroup() %>% select(Age,mass,jump_height_cm,m_rsi ,sex)

tbl_summary(table1, by = sex,
            label = list(Age ~ "Age (yrs)", mass ~ "Body Mass (kg)",
                         jump_height_cm ~ "Jump Height (cm)",
                         m_rsi ~ "Reactive Strength Index Modified")) %>%
                         add_difference() %>% modify_caption("Table 1")
```

```{r,}
df <- df %>%
  mutate(name = factor(name, levels = unique(name))) %>%
  mutate(athlete = paste("Athlete", as.numeric(factor(name)))) %>%
  mutate(athlete = factor(athlete, levels = paste("Athlete", 1:length(unique(name)))))

# Plot the data
p1 <- df %>% filter(athlete == "Athlete 1") %>%
  ggplot(aes(x = week, y = jump_height_cm, group = athlete, color = athlete)) +  # Group
by athlete for individual trajectories
  geom_point() +  # Plot points for each individual
  geom_smooth(method = "loess", se = TRUE, size = 1, aes(group = athlete)) +  # Add LOESS
smoothing
  theme_minimal() +  # Minimal theme
  labs(title = "Time Series Plot of Jump Height for Each Individual",
       x = "Week",
       y = "Jump Height (cm)") +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1)) +  # Adjust x-axis
text
  guides(color = guide_legend(reverse = FALSE))  # Reverse legend order if needed

p1
#ggsave("Jump_time.png")
```

```{r}
# Plot the data
p2 <- df %>%
  ggplot(aes(x = week, y = mass , group = athlete, color = athlete)) +  # Group by athlete
for individual trajectories
  geom_point() +  # Plot points for each individual
  geom_smooth(method = "loess", se = TRUE, size = 1, aes(group = athlete)) +  # Add LOESS
smoothing
  theme_minimal() +  # Minimal theme
  labs(title = "Time Series Plot of mass  for Each Individual",
       x = "Week",
       y = "mass  (kg)") +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1)) +  # Adjust x-axis
text
  guides(color = guide_legend(reverse = FALSE))  # Reverse legend order if needed

p2
ggsave("mass.png")
```

```{r}
p2 <- df %>%
  ggplot(aes(x = week, y = mass, group = athlete, color = athlete)) +  # Group by athlete
for individual trajectories
```

```
  geom_point() +  # Plot points for each individual
  geom_smooth(method = "loess", se = TRUE, size = 1, aes(group = athlete)) +  # Add LOESS
smoothing
  theme_minimal() +  # Minimal theme
  labs(title = "Time Series Plot of mass for Each Individual",
       x = "Week",
       y = "Mass (kg)") +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1)) +  # Adjust x-axis
text
  guides(color = "none") +  # Remove the color legend
  facet_wrap(~athlete)  # Create a separate plot for each athlete

p2
```

```{r}
df <- df %>% mutate(jump_height_cm = jump_height * 100)

df %>%
  ggplot(aes(x = week, y = jump_height_cm, group = sex, color = sex)) +  # Group by name
for individual trajectories
  geom_line() +   # Plot lines for each individual
  geom_point() +  # Plot points for each individual
  geom_smooth(method = "loess", se = FALSE,  size = 1, aes(group = sex)) +  # Add LOESS
smoothing
  theme_minimal() +  # Minimal theme
  labs(title = "Time Series Plot of Jump Height for Each Individual",
       x = "Week",
       y = "Jump Height (cm)") +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust = 1))  # Adjust x-axis
text

```

```{r}
p1 <- df %>%
  group_by(week, sex) %>%
  summarize(mean_jump_height = mean(jump_height_cm, na.rm = TRUE),
            se = sd(jump_height_cm, na.rm = TRUE) / sqrt(n())) %>%
  ggplot(aes(x = week, y = mean_jump_height, group = sex, color = sex)) +
  geom_line() +
  geom_errorbar(aes(ymin = mean_jump_height - se, ymax = mean_jump_height + se), width =
0.2) +
  geom_point() +
  theme_minimal() +
  #stat_smooth(method = "loess", se = FALSE, aes(group = sex)) +
  labs(title = "Time Series Plot of Mean Jump Height by Sex",
       x = "Week",
       y = "Mean Jump Height (cm)")
p1
```

```{r}
p2 <- df %>%
  ggplot(aes(x = date, y =  peak_propulsive_force, group = name, color = name)) +
  geom_line() +
  geom_point() +
  theme_minimal() +
  labs(title = "Total Peak Propulsive Force",
       x = "date",
       y = "N")  +
  #theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1)) +
  #scale_x_date(date_breaks = "1 month", datet_labels = "%b %Y", limits = date_range) +
  theme(legend.position = "none")
```

```
p3 <- df %>%
  ggplot(aes(x = date, y =  left_avg_propulsive_force, group = name, color = name)) +
  geom_line() +
  geom_point() +
  theme_minimal() +
  labs(title = "Left Leg Peak Propulsive Force",
       x = "date",
       y = "N") +
  #theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1)) +
 # scale_x_date(date_breaks = "1 month", date_labels = "%b %Y", limits = date_range) +
  theme(legend.position = "none")

p4 <- df %>%
  ggplot(aes(x = date, y =  right_avg_propulsive_force, group = name, color = name)) +
  geom_line() +
  geom_point() +
  theme_minimal() +
  labs(title = "Right Leg Peak Propulsive Force",
       x = "date",
       y = "N") +
 # theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1)) +
  #scale_x_date(date_breaks = "1 month", date_labels = "%b %Y", limits = date_range) +
  theme(legend.position = "none")
```

```{r, fig.height=10, warning=FALSE}
ggarrange(p2,p3,p4,ncol = 1)
```

```{r}
# Determine A as the max value between right_avg_propulsive_force and
left_avg_propulsive_force
# and B as the min value
df$A <- pmax(df$right_avg_propulsive_force, df$left_avg_propulsive_force)
df$B <- pmin(df$right_avg_propulsive_force, df$left_avg_propulsive_force)

# Calculate Index 9
df$Index1 <- abs(df$A - df$B) / df$A * 100

# Calculate Index 10
df$Index2 <- (df$A - df$B) / (df$A + df$B) * 100

# For Index 11, we use the original values
df$Index3 <- (45 - atan(df$B / df$A) * (180 / pi)) * 100 / 90

# Calculate Index 12, ensuring that A and B are greater than zero
df$Index4 <- ifelse(df$A > 0 & df$B > 0,
                    log(df$A / df$B) * 100, NA)

df$week <- as.numeric(as.Date(df$date) - min(as.Date(df$date))) / 7
```

```{r}
p5 <- df %>%
  ggplot(aes(x = week, y =  Index1, group = sex, color = sex)) +
  geom_line() +
  geom_point() +
  theme_minimal() + geom_smooth(method = "loess") +
  labs(title = "index 9",
       x = "week",
       y = "index") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1))
```

```
p6 <- df %>%
  ggplot(aes(x = week, y =  Index2, group = sex, color = sex)) +
  geom_line() +
  geom_point() +
  theme_minimal() + geom_smooth(method = "loess") +
  labs(title = "index 10",
       x = "week",
       y = "index") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1))

p7 <- df %>%
  ggplot(aes(x = week, y =  Index3, group = sex, color = sex)) +
  geom_line() +
  geom_point() +
  theme_minimal() + geom_smooth(method = "loess") +
  labs(title = "index 11",
       x = "week",
       y = "index") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1))

p8 <- df %>%
  ggplot(aes(x = week, y =  Index4, group = sex, color = sex)) +
  geom_line() +
  geom_point() +
  theme_minimal() + geom_smooth(method = "loess") +
  labs(title = "index 12",
       x = "week 12",
       y = "index") + theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, vjust = 0.9, hjust=1))
```

```{r, fig.height=10,fig.width=8, warning =FALSE}
ggarrange(p1,p5,p6,p7,p8, ncol=1)
```

```{r}
df_filter <- df %>% filter(Index1 < 30)
```

```{r, message=FALSE, fig.height=8}
ps_1 <- df %>%
  ggplot(aes(x = Index1, y = jump_height_cm)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 9",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_2 <- df %>%
  ggplot(aes(x = Index2, y = jump_height_cm)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 10",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_3 <- df %>%
  ggplot(aes(x = Index3, y = jump_height_cm)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 11",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_4 <- df %>%
  ggplot(aes(x = Index4, y = jump_height_cm)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 12",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ggarrange(ps_1,ps_2,ps_3,ps_4)
```

```

```{r, message=FALSE, fig.height=8}
ps_1 <- df %>%
  ggplot(aes(x = Index1, y = jump_height_cm, group = sex, color = sex)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 9",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_2 <- df %>%
  ggplot(aes(x = Index2, y = jump_height_cm, group = sex, color = sex)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 10",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_3 <- df %>%
  ggplot(aes(x = Index3, y = jump_height_cm, group = sex, color = sex)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 11",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ps_4 <- df %>%
  ggplot(aes(x = Index4, y = jump_height_cm, group = sex, color = sex)) +
  geom_point() + geom_smooth(method="loess") +  labs(title = "index 12",
       x = "Index",
       y = "Jump Height (cm)")+ theme_prism()

ggarrange(ps_1,ps_2,ps_3,ps_4)

```

Covariates

Mass

```{r}
#mass
model1m <- lmer(jump_height_cm ~ week*Index1  +
                  mass  + (1|name) + (1 + countermovement_depth|session_id), data=df)

#no mass
model1nom <- lmer(jump_height_cm ~ week*Index1  + (1|name) + (1 +
countermovement_depth|session_id), data=df)
```

```{r}
anova(model1m, model1nom)
```

age

```{r}
#age
model1a <- lmer(jump_height_cm ~ week*Index1  +
                  Age  + (1|name) + (1 + countermovement_depth|session_id), data=df)

#no age
model1noa <- lmer(jump_height_cm ~ week*Index1  + (1|name) + (1 +
countermovement_depth|session_id), data=df)
```

```{r}
anova(model1a, model1noa)
```
```

Jump Height

```{r}
model_all <- lmer(jump_height_cm ~ week * Index1*sex +
                    Index2*sex + Index3*sex  + Index4*sex  + Age +
                    mass  + (1 + countermovement_depth |name), data=df)
summary(model_all)
```

```{r, fig.height=10}
check_model(model_all)
```

```{r}
tab_model(model_all, show.aic = TRUE)
```

```{r}
plot_model(model_all)
```

AGE and SEX were REMOVED FRO MODELS SINCE THE COEFFICIENTS ARE CLOSE TO 0.

```{r}

#model0 <- lmer(jump_height_cm ~ week + Age + sex + mass + (1 +
countermovement_depth|name), data=df)

model1 <- lmer(jump_height_cm ~ week*Index1*sex  +
                  (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)


model2 <- lmer(jump_height_cm ~ week*Index2*sex  +
                 (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)

model3 <- lmer(jump_height_cm ~ week*Index3*sex +
              (1 + countermovement_depth | name) +
              (1 + countermovement_depth | session_id),
              data = df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))

model4 <- lmer(jump_height_cm ~ week*Index4*sex   +
                  (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
```

```{r}
tab_model(model1,model2,model3,model4, show.est = TRUE, collapse.ci = TRUE,p.style =
"stars",
  show.p = TRUE )
```

```{r,fig.width=8}
preds <-  predict_response(model2, terms = c("week","Index2", "sex"))
# Plotting the predictions

p1_index <- plot(preds, show_ci=FALSE,one_plot = TRUE,facets = FALSE) + theme_minimal() +
  labs(title = "Predicted Jump Height Over Time by Index 2",
       x = "Week",
       y = "Predicted Jump Height",
       color = "Index 2")
p1_index
```

```

```{r}
preds <-  predict_response(model3, terms = c("week","Index3", "sex"))
# Plotting the predictions

p2_index <- plot(preds, show_ci=FALSE,one_plot = TRUE,facets = FALSE) + theme_minimal() +
  labs(title = "Predicted Jump Height Over Time by Index 3",
       x = "Week",
       y = "Predicted Jump Height",
       color = "Index 3")
p2_index
```

```{r}
preds <-  predict_response(model3, terms = c("week","Index3", "sex"))
# Plotting the predictions

p2_index <- plot(preds, show_ci=FALSE,one_plot = TRUE,facets = FALSE) + theme_minimal() +
  labs(title = "Predicted Jump Height Over Time by Index 3",
       x = "Week",
       y = "Predicted Jump Height",
       color = "Index 3")
p2_index
```

```{r, fig.width=10}
ggarrange(p1,nrow = 2,
          (ggarrange(p1_index,p2_index,ncol = 2)),
          heights = c(0.7,1.3)
          )
```

```{r, fig.width=10,fig.height=5}
ggarrange(p1_index,p2_index,ncol = 2)
ggsave("NSCA_abstract.png")
```

```{r}
predictions <- ggpredict(model2, terms = c("week", "Index2 [0, 10, 20]", "sex"))

# Plot using ggplot2
ggplot(predictions, aes(x = x, y = predicted)) +
  geom_line() +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high, fill = group), alpha = 0.2) +
  facet_wrap(~facet, labeller = label_both) +
  labs(
    title = "Predicted Values of RSI",
    x = "Week",
    y = "RSI",
    color = "Sex",
    fill = "Sex"
  ) +
  theme_minimal()
```

```{r}
library(flexplot)
compare_performance(model1,model2, model3,model4)
```

```{r}
tab_model(model1,model2,model3,model4, show.aic = TRUE, p.style ="numeric_stars",
          collapse.ci = TRUE, digits = 3)
```
```

```{r,fig.height=6,message=False}
plot_models(model1, model2, model3, model4, show.values = TRUE,
  show.p = TRUE,
  spacing = 0.1) + ylim(-.010, .03) +
  theme_minimal()
```

```{r,fig.height=6,message=False}
p1_coef <- plot_model(model1, show.values = TRUE,
  show.p = TRUE,title = "Index 9",
  digits = 3,
  spacing = 1) + ylim(-.010, .03) +
  theme_prism()

p2_coef <- plot_model(model2, show.values = TRUE,
  show.p = TRUE,title = "Index 10",
  digits = 3,
  spacing = 1) + ylim(-.010, .03)+
  theme_prism()


p3_coef <- plot_model(model3, show.values = TRUE,
  show.p = TRUE,title = "Index 11",
  digits = 3,
  spacing = 1) + ylim(-.010, .03) +
  theme_prism()


p4_coef <- plot_model(model4, show.values = TRUE,
  show.p = TRUE,title = "Index 12",
  digits = 3,
  spacing = 1) + ylim(-.010, .03) +
  theme_prism()

```

```{r, fig.height=10, warning=FALSE}

ggarrange(p1_coef,p2_coef,p3_coef,p4_coef, ncol=2,nrow = 2)
ggsave("inter_coeff.png")
```

```{r}

#model0 <- lmer(jump_height_cm ~ week + Age + sex + mass + (1 +
countermovement_depth|name), data=df)

model1_noi <- lmer(jump_height_cm ~ week + Index1*sex  +
                     (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
model2_noi <- lmer(jump_height_cm ~ week + Index2*sex  +
                   (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
model3_noi <- lmer(jump_height_cm ~ week + Index3*sex +
                     (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
model4_noi <- lmer(jump_height_cm ~ week + Index4*sex   +
                     (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
```

```{r}
tab_model(model1,model2,model3,model4, show.aic = TRUE, p.style ="numeric_stars",
          collapse.ci = TRUE)
```

```

```{r}
library(ggeffects)
ggpredict(model1)
```

```{r}
predictions <- ggpredict(model1, terms = "week")

ggplot(predictions, aes(x = x, y = predicted)) +
    geom_line() +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.2) +
    labs(x = "Week", y = "Predicted Jump Height (cm)")
```

```{r}
interaction_predictions <- ggpredict(model1, terms = c("week", "Index1"))
ggplot(interaction_predictions, aes(x = x, y = predicted, color = group)) +
    geom_line() +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.1) +
    labs(x = "Week", y = "Predicted Jump Height (cm)", color = "Index1") +
    theme_minimal() +
    scale_color_brewer(palette = "Set1")
```

```{r}
triple_interaction_predictions <- ggpredict(model1, terms = c("week", "Index1", "sex"))

ggplot(triple_interaction_predictions, aes(x = x, y = predicted, color = group)) +
    geom_line() +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.1) +
    labs(x = "Week", y = "Predicted Jump Height (cm)", color = "Index1") +
    facet_wrap(~facet, scales = "free", ncol = 1) +
    theme_minimal() +
    scale_color_brewer(palette = "Set1")
```

```{r}
library(ggeffects)
ggpredict(model2)
```

```{r}
# Generate the triple interaction predictions
triple_interaction_predictions <- ggpredict(model1, terms = c("week", "Index1", "sex"))

# Plot the predictions
p_m1 <- ggplot(triple_interaction_predictions, aes(x = x, y = predicted, color = group)) +
    geom_line() +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.05) +  # Increase alpha
for visibility
    labs(x = "Week", y = "Predicted Jump Height (cm)", color = "Index1")  +  # Set legend
title for color
    facet_wrap(~facet, scales = "free", ncol = 1) +
    theme_minimal() +
    scale_color_brewer(palette = "Set1") + labs(color = "Index2")  +
    geom_point(alpha = 0.6)
p_m1
```

```{r}
# Generate the triple interaction predictions
triple_interaction_predictions <- ggpredict(model2, terms = c("week", "Index2", "sex"))
```
```

```
# Plot the predictions
p_m2 <- ggplot(triple_interaction_predictions, aes(x = x, y = predicted, color = group)) +
    geom_line() +
    geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.015) +  # Increase alpha
for visibility
    labs(x = "Week", y = "Predicted Jump Height (cm)", color = "Index2")  +  # Set legend
title for color
    facet_wrap(~facet, scales = "free", ncol = 1) +
    theme_minimal() +
    scale_color_brewer(palette = "Set1") + labs(color = "Index2")  +
    geom_point(alpha = 1)
p_m2
```

```{r, fig.height=12}
check_model(modelall)
```

RSI

```{r}

model1_m_rsi <- lmer(m_rsi ~ week*Index1*sex  +
                     (1 |name) + (1 |session_id), data=df,
                control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))
model2_m_rsi <- lmer(m_rsi ~ week*Index2*sex  +
                    (1 |name) + (1 |session_id), data=df,
                control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))

model3_m_rsi <- lmer(m_rsi ~ week*Index3*sex +
                (1  | name) +
                (1  | session_id),
                data = df,
                control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))

model4_m_rsi <- lmer(m_rsi ~ week*Index4*sex   +
                     (1 |name) + (1 |session_id), data=df,
                control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))
```

```{r}
tab_model(model1_m_rsi,model2_m_rsi,model3_m_rsi,model4_m_rsi,
          show.est = TRUE,
          collapse.ci = TRUE,
          p.style = "stars",
          show.p = TRUE,
          digits = 3)
```

Predicted values

Jump Height

```{r}
# Generate predictions over time for Index2 values of 0, 10, and 20 by sex
dat <- ggpredict(model3, terms = c("week [0:80]", "Index3 [0, 10, 20]", "sex"))

# Convert to a data frame for easier manipulation
dat_df <- as.data.frame(dat)

# Summarize predictions at the initial (week 0) and final (week 80) time points
```

```
summary_df <- dat_df %>%
  filter(x %in% c(0, 80)) %>%  # Filter for initial and final time points
  select(group, facet, x, predicted) %>%
  spread(x, predicted) %>%
  rename(week_0 = `0`, week_80 = `80`) %>%
  mutate(
    change_over_time = week_80 - week_0  # Calculate the change over time
  )

# Display the summarized changes
summary_df %>% filter(facet == "Male")
```

Jump Height

```{r}
# Generate predictions over time for Index2 values of 0, 10, and 20 by sex
dat <- ggpredict(model3_m_rsi, terms = c("week [0:80]", "Index3 [0, 10, 20]", "sex"))

# Convert to a data frame for easier manipulation
dat_df <- as.data.frame(dat)

# Summarize predictions at the initial (week 0) and final (week 80) time points
summary_df <- dat_df %>%
  filter(x %in% c(0, 80)) %>%  # Filter for initial and final time points
  select(group, facet, x, predicted) %>%
  spread(x, predicted) %>%
  rename(week_0 = `0`, week_80 = `80`) %>%
  mutate(
    change_over_time = week_80 - week_0  # Calculate the change over time
  )

# Display the summarized changes
summary_df %>% filter(facet == "Male")
```

```{r}
dat <- ggpredict(model3, terms = c("week", "Index3 [0, 10, 20]", "sex [Male]"))
p_res1 <- plot(dat, facets = TRUE,
  show_title  = FALSE) +
  labs(y = "Jump Height (cm)") +
  facet_wrap(~facet+group) + theme_classic()
#ggsave("index2predicted.png")
p_res1
```

```{r}
dat <- ggpredict(model3_m_rsi, terms = c("week", "Index3 [0, 10, 20]", "sex [Male]"))
p_res2 <-  plot(dat, facets = TRUE,
  show_title  = FALSE) +
  labs(y = "RSImod") +
  facet_wrap(~facet+group) + theme_classic()
#ggsave("index2predicted.png")
```

```{r}
ggarrange(p_res1,p_res2,nrow = 2,labels = c("A","B"))
ggsave("index3predicted.png")
```

#peak propulsive force

```{r}
# Determine A as the max value between right_avg_propulsive_force and
left_avg_propulsive_force
```

```
# and B as the min value

df$right_force_at_peak_propulsive_force <-
as.numeric(df$right_force_at_peak_propulsive_force)
df$left_force_at_peak_propulsive_force <-
as.numeric(df$left_force_at_peak_propulsive_force)


df$A_peak <- pmax(df$right_force_at_peak_propulsive_force,
df$left_force_at_peak_propulsive_force)
df$B_peak <- pmin(df$right_force_at_peak_propulsive_force,
df$left_force_at_peak_propulsive_force)

# Calculate Index 9
df$Index1_peak <- abs(df$A_peak - df$B_peak) / df$A_peak * 100

# Calculate Index 10
df$Index2_peak <- (df$A_peak - df$B_peak) / (df$A_peak + df$B_peak) * 100

# For Index 11, we use the original values
df$Index3_peak <- (45 - atan(df$B_peak / df$A_peak) * (180 / pi)) * 100 / 90

# Calculate Index 12, ensuring that A and B are greater than zero
df$Index4_peak <- ifelse(df$A_peak > 0 & df$B_peak > 0,
                    log(df$A_peak / df$B_peak) * 100, NA)

```

```{r}

#model0 <- lmer(jump_height_cm ~ week + Age + sex + mass + (1 +
countermovement_depth|name), data=df)

model1_peak <- lmer(jump_height_cm ~ week*Index1_peak*sex  +
                  (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
model2_peak  <- lmer(jump_height_cm ~ week*Index2_peak*sex  +
                 (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)

model3_peak  <- lmer(jump_height_cm ~ week*Index3_peak*sex +
              (1 + countermovement_depth | name) +
              (1 + countermovement_depth | session_id),
              data = df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))

model4_peak  <- lmer(jump_height_cm ~ week*Index4_peak*sex   +
                  (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df)
```

```{r}
tab_model(model1_peak ,model2_peak ,model3_peak ,model4_peak , show.est = FALSE)
```

```{r}

model1_peak_m_rsi <- lmer(m_rsi ~ week*Index1_peak*sex  +
                  (1 |name) + (1 |session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))
model2_peak_m_rsi <- lmer(m_rsi ~ week*Index2_peak*sex  +
                 (1 |name) + (1 |session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
```

```
100000)))

model3_peak_m_rsi <- lmer(m_rsi ~ week*Index3_peak*sex +
              (1  | name) +
              (1  | session_id),
              data = df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))

model4_peak_m_rsi <- lmer(m_rsi ~ week*Index4_peak*sex   +
                   (1 |name) + (1 |session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))
```

```{r}
tab_model(model1_peak_m_rsi ,model2_peak_m_rsi ,model3_peak_m_rsi ,model4_peak_m_rsi ,
show.est = TRUE)
```

```{r}
tab_model(model3 ,model3_peak, show.est = TRUE)
```

NSCA abstract df \<- df %\>% mutate(weight_N)

```{r}
df_female <- df %>% filter(sex == "Female")
model_fem <- lmer(jump_height_cm ~ week*peak_relative_braking_force  +
                    (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df_female,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))


summary(model_fem)
```

```{r}
model2 <- lmer(jump_height_cm ~ week*peak_relative_braking_force*sex  +
                    (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))


summary(model2)
```

```{r}
model3 <- lmer(peak_relative_propulsive_force ~ week*peak_relative_braking_force*sex  +
                    (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))


summary(model3)
```

```{r}
df$flight_time <- as.numeric(df$flight_time)
```

```{r}
```

```
model4 <- lmer(flight_time ~ week*peak_relative_braking_force*sex  +
                    (1 + countermovement_depth|name) + (1 +
countermovement_depth|session_id), data=df,
              control = lmerControl(optimizer = "bobyqa", optCtrl = list(maxfun =
100000)))


summary(model4)
```

```{r, fig.height=8}
library(ggplot2)
library(dplyr)
library(ggprism)
# Assuming df is your dataframe and it has columns 'peak_relative_braking_force',
'jump_height_cm', 'sex'

# Plot A: peak_relative_braking_force vs. jump_height_cm
plot_a <- df %>%
  ggplot(aes(x = peak_relative_braking_force, y = jump_height_cm, color = sex)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "A: Braking Force vs. Jump Height", x = "Peak Relative Braking Force", y =
"Jump Height (cm)") +
  theme_prism()

# Plot B: peak_relative_braking_force vs. peak_relative_propulsive_force
plot_b <- df %>%
  ggplot(aes(x = peak_relative_braking_force, y = peak_relative_propulsive_force, color =
sex)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "B: Braking Force vs. Propulsive Force", x = "Peak Relative Braking Force",
y = "Peak Relative Propulsive Force") +
  theme_prism()

# To display the plots side by side:
library(gridExtra)
grid.arrange(plot_a, plot_b, ncol = 1)

```

```{r}
tab_model(model2,model3,digits = 3)
```

```{r}
# Install required packages if not already installed
# Load libraries
library(xgboost)
library(dplyr)
library(tidyverse)
library(caret)
library(keras)
library(tensorflow)

```

```{r}
df <- df %>%
  mutate(Index3_cat = cut(Index3, breaks = c(-Inf, 2.5, 7.5, 12.5, 17.5, Inf),
                          labels = c(0, 5, 10, 15, 20)))
df$Index3_cat <- as.numeric(as.character(df$Index3_cat))  # Convert to numeric
feature_matrix <- as.matrix(df %>% select(-jump_height_cm, -Index3))  # Remove old
continuous Index3
```

```
label_vector <- df$jump_height_cm
table(df$Index3_cat, df$sex)

```

```{r}
# Load necessary libraries
library(xgboost)
library(caret)  # For cross-validation

# Convert data to matrix format for XGBoost
feature_matrix <- as.matrix(df %>% select(-jump_height_cm))
label_vector <- df$jump_height_cm

# Set up 10-fold cross-validation
set.seed(42)
train_control <- trainControl(
  method = "cv",  # Cross-validation
  number = 10,  # 10 folds
  verboseIter = TRUE
)

tune_grid <- expand.grid(
  nrounds = 200,  # More boosting rounds
  eta = 0.03,  # Lower learning rate for finer adjustments
  max_depth = 10,  # Deeper trees to capture nonlinearity
  gamma = 0.05,  # Less regularization
  colsample_bytree = 0.9,
  min_child_weight = 1,
  subsample = 0.95
)

# Train XGBoost with 10-fold CV
xgb_cv_model <- train(
  x = feature_matrix,
  y = label_vector,
  method = "xgbTree",
  trControl = train_control,
  tuneGrid = tune_grid
)

# View best model performance
print(xgb_cv_model)

```

```{r}
# Ensure Index3 (asymmetry) is removed before expanding grid
asymmetry_test <- df %>%
  group_by(sex) %>%
  slice_head(n = 1) %>%  # Take one example per sex
  ungroup() %>%
  select(-jump_height_cm, -Index3) %>%  # Remove Index3 before expanding
  expand_grid(Index3 = c(0, 5, 10, 15, 20))  # Test different asymmetry values

# Ensure column order matches training data
feature_names <- colnames(feature_matrix)
asymmetry_test <- asymmetry_test[, feature_names]

# Convert to matrix for XGBoost prediction
asymmetry_matrix <- as.matrix(asymmetry_test)

# Make predictions
asymmetry_test$Predicted_Jump_Height <- predict(xgb_cv_model, asymmetry_matrix)
```

```
# View results
print(asymmetry_test)

```

```{r}
# Load ggplot2 for visualization
library(ggplot2)

# Plot asymmetry vs. predicted jump height
ggplot(asymmetry_test, aes(x = Index3, y = Predicted_Jump_Height, color = as.factor(sex)))
+
  geom_line(size = 1.2) +
  geom_point(size = 3) +
  geom_ribbon(aes(ymin = Predicted_Jump_Height - 1.96 * 1.556, ymax =
Predicted_Jump_Height + 1.96 * 1.556, fill = as.factor(sex)), alpha = 0.2) +  # 95% CI
using RMSE
  labs(
    title = "Predicted Jump Height vs. Asymmetry (0%, 5%, 10%, 15%, 20%) with 95% CI",
    x = "Asymmetry (%)",
    y = "Predicted Jump Height (cm)",
    color = "Sex",
    fill = "Sex"
  ) +
  theme_bw() +
  facet_grid(~sex)


```